NER SOFTWARE · AI TOOLING

Delphi

Where your agent's context comes from,
and how improvements get back to everyone.

Delphi 1.0.0 (main 33a2141), October 2026 · github.com/Northeastern-Electric-Racing/Delphi

THE MODEL

A workspace is a folder on main, and a branch you clone

main every workspace, as a folder

```
software/
  application-software/
    argos/
      workspaces/
        argos-dev/
          CLAUDE.md    workspace.yml
          .claude/     docs/
        experimental/
```

.delphi/ the scripts, at main's root

refresh
CI

propose
CI opens a PR

ws/s.app.argos.argos-dev root = folder

```
CLAUDE.md
workspace.yml
.claude/
docs/
```

same files, now at the top

.claude/skills/… inherited from folders above

.delphi/ copied from main's root by CI

You clone this branch and run `.delphi/setup.sh once`.

folder `software/application-software/argos/~~workspaces/~~argos-dev/`

branch `ws/s.app.argos.argos-dev`

name: each folder's short name, joined by dots
(.delphi/names.sh lists them)

WORKSPACE.YML

One small file says what a workspace pulls in

workspace.yml

```yaml
name: s.app.argos.argos-dev
harness: claude-code
plugins:
  - delphi   # default, added by CI
park:
  - pictures
  - .playwright-cli
repos:
  argos: …/Argos.git
  odyssey-definitions: …/Odyssey-Definitions.git
references-git:
  socketioxide: …/socketioxide.git socketioxide-v0.18.7
  zenoh: …/zenoh.git 1.10.1
workspaces:
  - main
```

`name:` Which workspace this is
its dotted name; the branch is ws/<name>

`harness:` Which agent it's for
claude-code: CI writes .claude/settings.json

`plugins:` Plugin sets from folders above
CI adds the defaults; setup.sh installs them

`park:` Folders that switch with it
moved to .delphi/parked/<name>/ while you're away

`repos:` Code you change
store in repos/<name>, checkouts in worktrees/

`references-git:` Dependency source to read
references/<name>/, shallow, at the pin

`workspaces:` Other workspaces to read
a clone on its branch at workspaces/<name>/

It's shared: change it on a branch, PR it into ws/<name>, and everyone re-runs `.delphi/setup.sh`.

WORKSPACE.YML → DISK

Each key turns into something in your clone

IN WORKSPACE.YML	READ BY · WHEN	YOU GET
`harness:` `plugins:`	**`ci/plugins.sh`** CI, on every sync	**`.claude/settings.json`** enables delphi@delphi; setup.sh installs it
`repos:`	**`.delphi/setup.sh`** you, after a clone or an edit	**`repos/argos/ + worktrees/argos/develop/`** one bare store; a worktree per branch
`references-git:`	**`.delphi/setup.sh`** every run, re-pins	**`references/zenoh/ at 1.10.1`** shallow, detached, pushes disabled
`workspaces:`	**`.delphi/setup.sh`** every run: pull + its setup	**`workspaces/main/`** a full clone, on main, tracking origin
`name:` `park:`	**`.delphi/park.sh`** git hook, on every checkout	**`.delphi/parked/<name>/`** holds a workspace's folders while you're elsewhere

Everything setup.sh makes is git-ignored, so code never shows up as a workspace change.

WHERE THINGS COME FROM

Every file in your clone has one source

argos-dev/ your clone of ws/s.app.argos.argos-dev

- `CLAUDE.md`
- `workspace.yml`
- `docs/CONTEXT.md`
- `.claude/skills/dependabot-cleanup/`
- `.claude/skills/implement/ commit/ …` *from workspace-inherit/*
- *`plugin delphi · /ws-delphi`* *installed by setup.sh*
- `.claude/settings.json`
- `.delphi/`
- `.github/workflows/delphi.yml`
- `repos/ worktrees/`
- `references/ workspaces/`
- `.claude/settings.local.json`
- `.journal/`

The workspace's own
from its folder on main
change **PR into ws/<name>**

Delphi's machinery
from main's root, synced to every ws/*
change **PR to main only**

Generated by CI
from ci/plugins.sh, from workspace.yml
change **never by hand: edit plugins:**

Inherited from folders above
from workspace-inherit/, plugins/ above it
change **PR to main; CI or setup.sh brings it**

Made by setup.sh
from workspace.yml, via setup.sh
change **edit workspace.yml, re-run**

Yours alone
from you; git-ignored
change **nothing to share**

WHERE THINGS COME FROM

Three ways context from above reaches a workspace

main folders above argos-dev

```
software/
  workspace-inherit/  → journal, switch-workspace, …
  application-software/
    workspace-inherit/  → commit, open-pr, …
    argos/
      workspace-inherit/  → implement, run-local
      workspace-defaults/  → copied at creation
      README · AUTHORING.md
      workspaces/
        argos-dev/
```

workspace-defaults/

Copied once, when the workspace is made

tools/new-workspace.sh copies templates/workspace/ plus the project's workspace-defaults/. Then the copy is the workspace's own.

workspace-inherit/

One copy on main, on every branch below

Each sync, CI writes it onto every ws/* under that folder; an edit there is undone (list it under opt-out: to keep your own). Plugin sets only carry what files can't: hooks, the delphi mod.

workspaces/main/

Read when needed, not edited from here

With - main under workspaces:, setup.sh keeps a clone of main, pulled on every run: the project folder is at workspaces/main/software/…/argos/.

WORKFLOW 1 / 2

A code change: skills from Delphi, code to Argos

1 Clone + setup

`git clone -b ws/…`
`.delphi/setup.sh`

→

2 /implement

worktree from
origin/develop, test-first

→

3 /run-local

serves that
worktree's client

→

4 /commit

`#533 - add CSV`
`upload endpoint`

→

5 /open-pr

lint + checks,
draft PR → develop

argos-dev/ your clone

- **.claude/skills/implement/ …** — *the how-to, inherited from argos/*
- **repos/argos/** — *one bare store*
- **worktrees/argos/develop/** — *mirror of develop: run, don't edit*
- **worktrees/argos/533-csv-upload-download-rules/** — **your branch**

Git-ignored: the code never lands in Delphi.

push →

Code ships through
the code repo's own PRs.

GitHub · Argos

draft PR → develop

`Closes #533`

WORKFLOW 2 / 2

A context change: fix it once, every clone gets it

IN YOUR WORKSPACE

1 Notice a gap

a skill missed a step:
park it with /journal

2 Fix it on a branch

branch off ws/<name>; edit
.claude/skills/, CLAUDE.md, docs/

3 PR into ws/<name>

reviewed like code,
then merged

ON MAIN, THEN BACK TO EVERY WORKSPACE

4 CI proposes

`propose/<name> → PR to main`
labelled by: ai; check.sh runs

5 Merge into main

merge commit or squash,
never rebase

6 CI refreshes

every ws/* branch gets it;
you: git merge origin/ws/<name>

Shared by many workspaces?

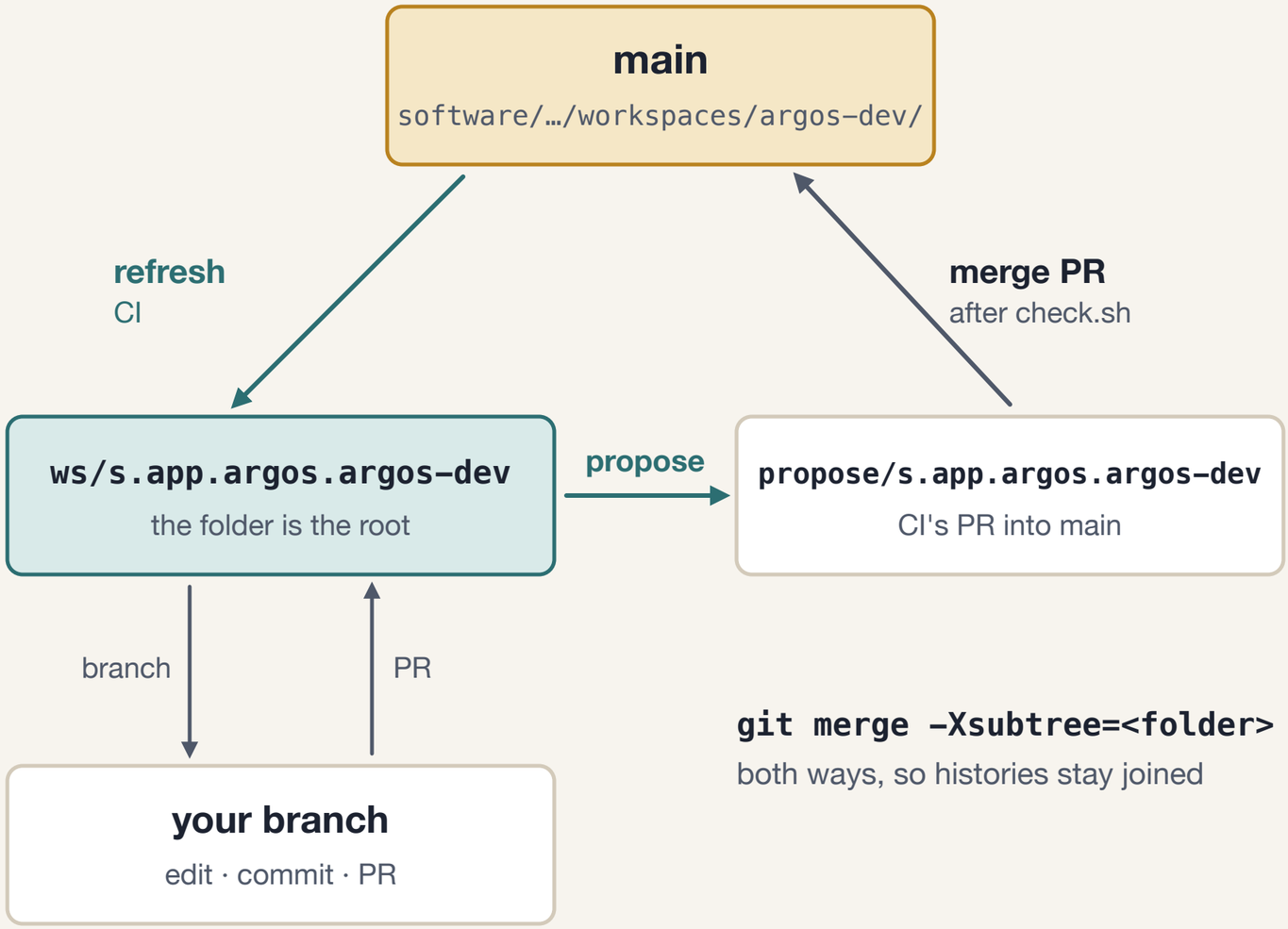
Put it in a folder's workspace-inherit/ instead: one PR to main, and every workspace below gets it.

Conflict with main?

CI lists the files and skips that workspace; fix it in a PR into ws/<name>.

UNDER THE HOOD

Two CI merges keep the folder and the branch in step



`git merge -Xsubtree=<folder>`
both ways, so histories stay joined

Refresh

Each push to main is merged into every ws/<name>, folder becoming root.

Propose

Merged workspace changes become a PR to main from propose/<name>.

One copy of the machinery

Sync puts main's .delphi/ and inherited files on every branch, and regenerates settings.json.

Conflicts are reported

CI lists the files and skips that workspace; a person resolves it.

WORKSPACE.YML · IN PRACTICE

Need more in your workspace? Edit workspace.yml

workspace.yml (e.g.)

```
  repos:
    argos: …/Argos.git
+   nero: …/Nero-2.0.git
  references-git:
    zenoh: …/zenoh.git 1.10.1
  workspaces:
    - main
+   - s.app.nero.nero-experimental
```

after setup.sh, git-ignored:

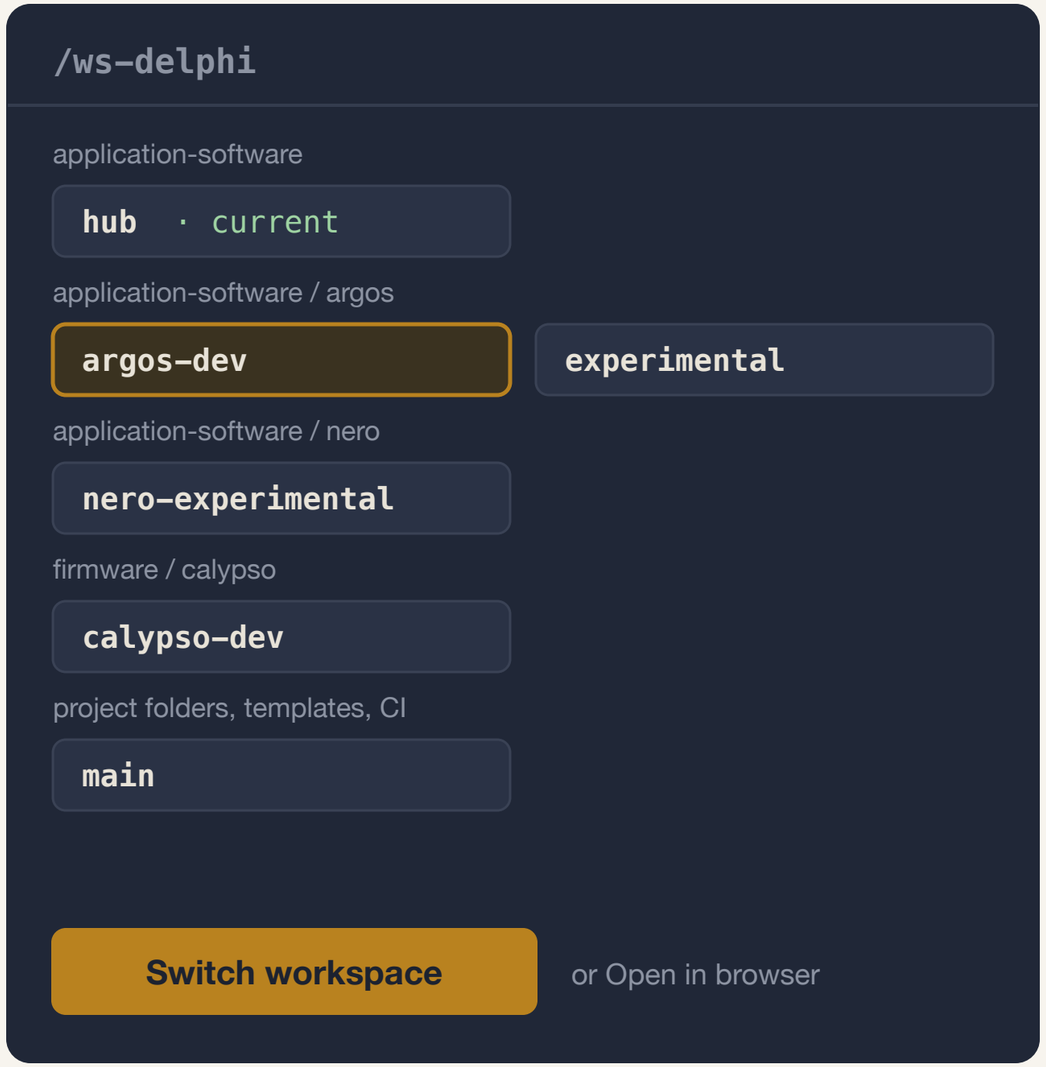
`worktrees/nero/develop/` `workspaces/s.app.nero.nero-experimental/`

1. **Edit workspace.yml**
   on a branch; the setup-workspace skill knows the format
2. **Re-run .delphi/setup.sh**
   new folders appear; linked clones get set up too
3. **PR it into ws/<name>**
   teammates get it on their next setup.sh
4. **Plugin sets: CI does the rest**
   after the merge it regenerates .claude/settings.json

Just looking? Add the entry, run setup, then `git restore workspace.yml`. To work in another workspace, switch (next slide).

SWITCHING

One clone, any workspace: /ws-delphi



1. **Checks you're clean**
   refuses while tracked files have changes
2. **Parks this workspace**
   its folders + park: list go to .delphi/parked/<name>/
3. **Sets up the new one**
   migrations, repos, references, its plugins
4. **No restart**
   skills reload; the next prompt gets the new CLAUDE.md

Without the mod: `.delphi/switch.sh <name>|main`

List them: `.delphi/switch.sh --list`

Branches of one workspace move nothing; parked work is ignored, not lost.

CHEAT SHEET

Where do I change…?

I WANT TO…	EDIT	IT LANDS BY
Fix a skill, CLAUDE.md or docs	`.claude/skills/…  (in the clone)`	PR into ws/<name>
Pull in a repo, dependency, workspace	`workspace.yml`	PR into ws/<name>; re-run setup
Turn on another plugin set	`workspace.yml  plugins:`	CI regenerates settings.json
Share a skill with many workspaces	`software/…/workspace-inherit/`	PR to main; CI writes it below
Change what new workspaces start with	`…/<project>/workspace-defaults/`	PR to main (switch to main)
Change setup, switch, park scripts	`.delphi/  on main`	PR to main + version bump
Change Argos code	`worktrees/argos/<branch>/`	Argos PR into develop
Personal settings	`.claude/settings.local.json`	nowhere: git-ignored

The rule: change a thing where it comes from.

TRY IT

Three commands to a ready agent

```
git clone -b ws/s.app.argos.argos-dev \
    https://github.com/Northeastern-Electric-Racing/Delphi.git delphi
cd delphi && .delphi/setup.sh
claude                    # then /implement, /open-pr, /ws-delphi …
```

Re-run setup after changing workspace.yml. **Switch** with `/ws-delphi` or `.delphi/switch.sh <name>|main`

Read more on main: README.md · docs/design.md · docs/goals.md